

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.

14-mashg‘ulot. Pythonda OOP asoslari.

O‘quv savollari:

1. OOP asoslari.
2. Klass va obyekt
3. `__init__()` (konstruktor) va `__del__()` (destruktor) metodlari.

1. OOP asoslari.

OOP nima?

OOP (Object-Oriented Programming) — obyektga yo‘naltirilgan dasturlash degan ma'noni anglatadi. OOP'ning asosiy g‘oyasi — dasturning ishlashini "obyektlar" orqali tashkil qilish. Obyektlar esa ma'lum bir xususiyatlarga (atributlarga) va xatti-harakatlarga (metodlarga) ega bo‘ladi.

OOP ning asosiy tushunchalari

- **Klass** — bu obyektlarni yaratish uchun shablon.
- **Obyekt** — bu klassdan yaratilgan aniq nusxadir.
- **Atributlar** — bu obyektning xususiyatlari.
- **Metodlar** — bu obyektning xatti-harakatlarini belgilovchi funksiyalar.
- **Inkapsulyatsiya** — ma'lumotlarni va metodlarni bir joyda jamlash va tashqaridan boshqarilishini cheklash.
- **Meros** — bir klassning metodlari va atributlarini boshqa klasslarga o‘tkazish.
- **Polimorfizm** — bir xil nomdagi metodlarning turli obyektlarda turli tarzda ishlashi.
- **Abstraksiya** — foydalanuvchidan ichki murakkabliklarni yashirish va faqat kerakli qismlarni ko‘rsatish.

OOP ning asosiy tamoyillari (prinsiplari) quyidagilardan iborat:

- **Inkapsulyatsiya** — ma'lumotlarni yashirish va ularga faqat metodlar orqali kirish.
- **Abstraksiya** — ichki murakkablikni yashirish va foydalanuvchiga kerakli interfeysni taqdim etish.
- **Meros** — mavjud klassdan yangi klass yaratish va metodlarni qayta ishlatish.
- **Polimorfizm** — bir xil metodlarning turli obyektlarda turlicha ishlashi.

Python tilida **OOP (Object-Oriented Programming)** yordamida dasturda **klasslar** va **obyektlar** tuziladi.

- **Klass** — bu ma'lum bir turdagi obyektlarni yaratish uchun shablondir. Klassda obyektlarning atributlari va metodlari belgilanishi mumkin.
- **Obyekt** — klassdan yaratilgan aniq bir nusxa.

2. Klass va obyekt

Klass (Class)

Klass — bu obyektlarni yaratish uchun shablon yoki namunadir. Klassda obyektning **atributlari** (xususiyatlari) va **metodlari** (harakatlari) ta'riflanadi. Klass yordamida biz bir xil turdagi obyektlarni yaratish va ularni boshqarishimiz mumkin.

Klassning Sintaksisi

```
class KlassNomi:
    # Klass atributlari
    atribut1 = "some value"
    atribut2 = 123

    # Klass metodlari
    def metod1(self):
        print("Bu metod1.")

    def metod2(self):
        print("Bu metod2.")
```

Yuqoridagi misolda:

- **KlassNomi** — bu klass nomi.
- **atribut1 va atribut2** — bu klass atributlari bo'lib, ular hamma obyektlar uchun umumiydir.
- **metod1 va metod2** — bu klass metodlari, ular obyektlarning harakatlarini ta'riflaydi.

Obyekt (Object)

Obyekt — bu klassdan yaratilgan konkret nusxa bo'lib, uning o'ziga xos atributlari va metodlari bor. Klassni **instanciyalash** orqali obyekt yaratiladi. Obyektlar bir xil klassga tegishli bo'lsalar ham, ular o'ziga xos atributlarga ega bo'lishi mumkin.

Obyekt yaratish

```
# Klassdan obyekt yaratish
obyekt1 = KlassNomi() # Klassdan obyekt yaratildi
obyekt2 = KlassNomi() # Yana bir obyekt

# Klass metodlarini chaqirish
obyekt1.metod1() # Bu metod1.
obyekt2.metod2() # Bu metod2.
```

Yuqoridagi misolda:

- obyekt1 va obyekt2 — KlassNomi klassidan yaratilgan obyektlar.
- Har bir obyekt o'ziga xos bo'lib, metodlarni chaqiradi.

Klass va Obyekt Atributlari

Klass atributlari — bu klassga tegishli bo'lgan atributlar, ular barcha obyektlar uchun umumiydir.

Obyekt atributlari — bu har bir obyektga xos atributlar. Obyekt yaratishdan keyin, ularning qiymatlari o'zgarishi mumkin.

Talaba misoli:

- **Klass atributlari:** Klassga umumiy bo'lgan atributlar (masalan, universitet nomi yoki fakultet).
- **Obyekt atributlari:** Har bir talaba uchun alohida bo'lgan atributlar (masalan, ism, yosh, kurs).

```
class Talaba:
    # Klass atributlari (bular barcha talabalarga umumiy)
    universitet = "O'zbekiston Milliy Universiteti"
    fakultet = "Matematika"

    # Obyekt atributlari (har bir talaba uchun alohida)
    def __init__(self, ism, yosh, kurs):
        self.ism = ism # Talabaning ismi (obyektga xos)
        self.yosh = yosh # Talabaning yoshi (obyektga xos)
        self.kurs = kurs # Talabaning kursi (obyektga xos)

    def info(self):
        print(f"{self.ism} {self.universitet}da {self.fakultet} fakultetida {self.kurs}-kurs talabasi, {self.yosh} yoshda.")

# Obyektlar yaratish
talaba1 = Talaba("Ali", 20, 2)
talaba2 = Talaba("Vazira", 22, 3)

# Obyekt metodlarini chaqirish
talaba1.info() # Ali O'zbekiston Milliy Universitetida Matematika fakultetida 2-kurs talabasi, 20 yoshda.
talaba2.info() # Vazira O'zbekiston Milliy Universitetida Matematika fakultetida 3-kurs talabasi, 22 yoshda.
```

Klass metodlarini chaqirish

Klassda e'lon qilingan metodlar obyektlar tomonidan chaqiriladi.

```
class Avto:
    # Klass atributi
    marka = "Toyota"

    # Metodlar
```

```

def info(self):
    print(f"Bu avto {self.marka} marka va {self.yil} yil.")

def yilni_sozla(self, yil):
    self.yil = yil

# Obyekt yaratish
avto1 = Avto()
avto2 = Avto()

# Obyekt metodlarini chaqirish
avto1.yilni_sozla(2018)
avto2.yilni_sozla(2020)

avto1.info() # Bu avto Toyota marka va 2018 yil.
avto2.info() # Bu avto Toyota marka va 2020 yil.

```

- `info()` — bu metod avtoning markasi va yilini chiqaradi.
- `yilni_sozla()` — bu metod, avto obyektining yilini sozlaydi.

3. `__init__()` (Konstruktor) va `__del__()` (Destruktor) metodlari

`__init__()` va `__del__()` metodlari Python'dagi **maxsus metodlardir**. Ular **obyektlar bilan ishlash** jarayonida avtomatik ravishda chaqiriladi va obyektlarni yaratish yoki o'chirish kabi muhim vazifalarni bajaradi.

1. `__init__()` (Konstruktor) Metodi

- `__init__()` — bu **konstruktor** metodidir, ya'ni obyekt yaratishda avtomatik tarzda chaqiriladigan metod.
- `__init__()` metodini klassda belgilash orqali, yangi obyekt yaratilganda uning boshlang'ich holatini belgilashimiz mumkin.
- Bu metod **obyekt yaratish** uchun ishlatiladi va **obyektni boshlang'ich qiymatlari bilan to'ldiradi**.
- `self` parametri — metodni chaqirgan obyektni bildiradi va uni metodda ishlatish imkonini beradi.

Misol:

```

class Talaba:
    def __init__(self, ism, yosh): # Konstruktor metodi
        self.ism = ism # Obyektning ism atributi
        self.yosh = yosh # Obyektning yosh atributi

    def salomlash(self):
        print(f"Salom, men {self.ism} va yoshim {self.yosh}.")

# Obyekt yaratish
t1 = Talaba("Ali", 20) # `__init__` metodini chaqirish
t2 = Talaba("Vazira", 22) # `__init__` metodini chaqirish

```

```
t1.salomlash() # Salom, men Ali va yoshim 20.  
t2.salomlash() # Salom, men Vazira va yoshim 22.
```

2. `__del__()` (Destruktor) Metodi

- `__del__()` — bu **destruktor** metodidir, ya'ni obyekt o'chirilganida avtomatik tarzda chaqiriladi.
- `__del__()` metodini ishlatish orqali obyektни o'chirish jarayonida bajariladigan ishlarni aniqlash mumkin. Masalan, fayllarni yopish, resurslarni tozalash va hokazo.
- `__del__()` metodining chaqirilishi **obyekt o'chirilganda** amalga oshiriladi.

Misol:

```
class Talaba:  
    def __init__(self, ism, yosh):  
        self.ism = ism  
        self.yosh = yosh  
        print(f"{self.ism} talaba yaratildi.")  
  
    def salomlash(self):  
        print(f"Salom, men {self.ism} va yoshim {self.yosh}.")  
  
    def __del__(self): # Destruktor metodi  
        print(f"{self.ism} talaba o'chirildi.")  
  
# Obyekt yaratish  
t1 = Talaba("Ali", 20)  
t1.salomlash() # Salom, men Ali va yoshim 20.  
  
# Obyektni o'chirish  
del t1 # `__del__()` metodi chaqiriladi
```

`__init__()` va `__del__()` Metodlarining Farqlari:

Xususiyat	<code>__init__()</code> (Konstruktor)	<code>__del__()</code> (Destruktor)
Vaqt	Obyekt yaratishda chaqiriladi.	Obyekt o'chirilganda chaqiriladi.
Funksiyasi	Obyektni boshlang'ich qiymat bilan to'ldirish.	Obyektni o'chirayotganda resurslarni tozalash.
Chaqirilishi	Yangi obyekt yaratganda avtomatik chaqiriladi.	del kalit so'zi orqali chaqiriladi.

Amaliy mashqlar

Misol 1: Kitoblar Ma'lumotlar Bazasi

Kitoblarni boshqaradigan dastur tuzilsin. Dasturda har bir kitobning **nomi**, **muallifi** va **yili** bo'ladi. Shuningdek, kitoblar to'plamini yaratish va ularni o'chirishni boshqarish kerak.

- **Konstruktor** (`__init__()`) yordamida kitobni yaratilsin va uni boshlang'ich ma'lumotlar bilan to'ldirilsin.
- **Destruktor** (`__del__()`) yordamida kitobni o'chirilsin, kitobning resurslari tozalansin.

Kod:

```
class Kitob:
    def __init__(self, nomi, muallifi, yil):
        self.nomi = nomi # Kitobning nomi
        self.muallifi = muallifi # Kitobning muallifi
        self.yil = yil # Kitobning nashr yili
        print(f"{self.nomi} kitobi yaratildi.")

    def __del__(self):
        print(f"{self.nomi} kitobi o'chirildi.")

# Kitoblar yaratish
kitob1 = Kitob("Python Dasturlash", "John Smith", 2020)
kitob2 = Kitob("OOP Asoslari", "Jane Doe", 2019)

# Kitoblar haqida ma'lumot
print(f"Kitob: {kitob1.nomi}, Muallif: {kitob1.muallifi}, Yil: {kitob1.yil}")
print(f"Kitob: {kitob2.nomi}, Muallif: {kitob2.muallifi}, Yil: {kitob2.yil}")

# Kitobni o'chirish
del kitob1 # `__del__()` metodi chaqiriladi
del kitob2 # `__del__()` metodi chaqiriladi
```

Chiqish:

```
Python Dasturlash kitobi yaratildi.
OOP Asoslari kitobi yaratildi.
Kitob: Python Dasturlash, Muallif: John Smith, Yil: 2020
Kitob: OOP Asoslari, Muallif: Jane Doe, Yil: 2019
Python Dasturlash kitobi o'chirildi.
OOP Asoslari kitobi o'chirildi.
```

Misol 2: Bank Hisob Raqami

Bankdagi **hisob raqamlarini** yaratish va yopish jarayoni dasturi tuzilsin. Har bir hisob raqamining o'z **mijoz ismi**, **balans** (pul miqdori), va **raqami** bo'ladi. Hisob raqamlarini yaratish va yopish uchun konstruktor va destruktur metodlaridan foydalanilsin.

- **Konstruktor** (`__init__()`) yordamida mijozning hisob raqamini yaratib, balansni belgilansin.

- **Destruktor** (`__del__()`) yordamida hisob raqamining yopilish jarayonida uni tozalansin (masalan, balansni yangilash yoki yopish).

Kod:

```
class HisobRaqami:
    def __init__(self, mijoz_ismi, raqam, balans):
        self.mijoz_ismi = mijoz_ismi # Mijozning ismi
        self.raqam = raqam # Hisob raqami
        self.balans = balans # Hisob balansi
        print(f"{self.mijoz_ismi} uchun hisob raqami yaratildi.")

    def __del__(self):
        print(f"{self.mijoz_ismi} hisob raqami yopildi.")

    def balans_tekshirish(self):
        print(f"{self.mijoz_ismi} hisob raqamidagi balans: {self.balans} so'm.")

    def pul_yechish(self, summa):
        if summa <= self.balans:
            self.balans -= summa
            print(f"{self.mijoz_ismi} hisobidan {summa} so'm yechildi.")
        else:
            print(f"{self.mijoz_ismi} hisobida yetarli mablag' yo'q.")

# Hisob raqamlarini yaratish
hisob1 = HisobRaqami("Ali", "UZ123456789", 1000)
hisob2 = HisobRaqami("Vazira", "UZ987654321", 1500)

# Hisob raqamlariga murojaat qilish
hisob1.balans_tekshirish() # Ali hisob raqamidagi balans: 1000 so'm
hisob1.pul_yechish(200) # Ali hisobidan 200 so'm yechildi.
hisob1.balans_tekshirish() # Ali hisob raqamidagi balans: 800 so'm

# Hisobni yopish
del hisob1 # `__del__()` metodi chaqiriladi
del hisob2 # `__del__()` metodi chaqiriladi
```

Chiqish:

```
Ali uchun hisob raqami yaratildi.
Vazira uchun hisob raqami yaratildi.
Ali hisob raqamidagi balans: 1000 so'm.
Ali hisobidan 200 so'm yechildi.
Ali hisob raqamidagi balans: 800 so'm.
Ali hisob raqami yopildi.
Vazira hisob raqami yopildi.
```

Masala #0202

Xotira 16 MB Vaqt 1000 ms Qiyinchiligi 7 % ☆

★★★★☆ 3.9 (Baholar 65)

Muallif: Hojiyev Sunatillo

14

Gugurt donalari va raqamlar



Yuqoridagi rasmda har bir raqamni gugurt donalari yordamida ifodalanishi ko'rsatilgan.

Kiruvchi ma'lumotlar:

Kirish faylida bitta butun son beriladi, $N(0 \leq N \leq 10^9)$

Chiquvchi ma'lumotlar:

Chiqish faylida bitta butun son, berilgan N sonini gugurt donalari yordamida ifodalash uchun jami nechta gugurt donasi kerak bo'lishini chop eting!

Misollar

#	input.txt		output.txt	
1	137		10	
2	379		14	

```
class MatchstickCalculator:
    matchsticks = {'0': 6, '1': 2, '2': 5, '3': 5, '4': 4, '5': 5, '6': 6, '7': 3,
                  '8': 7, '9': 6}

    def __init__(self, number):
        self.number = number

    def calculate_matchsticks(self):
        return sum(self.matchsticks[digit] for digit in str(self.number))
# Foydalanuvchidan sonni olish va hisoblash
N = int(input())
print(MatchstickCalculator(N).calculate_matchsticks())
```


Sichqon va Mushuklar



Ikkita mushuk va bitta sichqon to'g'ri chiziqli bo'ylab turli xil nuqtalarda joylashgan. Sizga ularning boshlang'ich nuqtalari berilgan. Sichqon pishloq iste'mol qilish bilan ovora bo'lganligi uchun mushuklarni ko'rmagan, shuning uchun u mushuklardan qochmasdan o'z o'rnidan qimirlamaydi. Ikkala mushukning tezligi bir xil, qaysi mushuk sichqonning oldiga birinchi yetib kelsa sichqonni o'sha mushuk qo'lga kiritadi. Agar ikkala mushuk ham sichqonni oldiga bir vaqtda yetib kelishsa sichqonni ustiga o'zaro tortishib qolishadi va paytdan foydalangan holda sichqon qochib qoladi. Sizning vazifangiz:

- Agar birinchi mushuk sichqonni qo'lga kiritsa "1-mushuk"
- Agar ikkinchi mushuk sichqonni qo'lga kiritsa "2-mushuk"
- Agar sichqon qochib qolsa "sichqon"

deb xabar chiqarishdan iborat.

Kiruvchi ma'lumotlar:

Kirish faylining yagona satrida 3 ta butun son, A, B va $C(1 \leq A, B, C \leq 100)$ sonlari berilgan, bu sonlar mos ravishda 1-mushukning, 2-mushukning va sichqonning boshlang'ich nuqtalari hisoblanadi.

Chiquvchi ma'lumotlar:

So'ralgan javobni chop eting!

Misollar

#	input.txt	output.txt
1	1 2 3	2-mushuk
2	1 3 2	sichqon

```
class CatchMouse:
    def __init__(self, cat1, cat2, mouse):
        self.cat1, self.cat2, self.mouse = cat1, cat2, mouse

    def catch(self):
        d1, d2 = abs(self.cat1 - self.mouse), abs(self.cat2 - self.mouse)
        return "1-mushuk" if d1 < d2 else "2-mushuk" if d2 < d1 else "sichqon"

A, B, C = map(int, input().split())
print(CatchMouse(A, B, C).catch())
```